

CSE 4102: Computer Graphics and Image Processing Laboratory

**MOONLIT FOREST CAMPSITE:
A REAL-TIME PROCEDURAL 3D SCENE
USING C++ AND OPENGL**

Submitted by

MD Ruhan Ahsan Sarnav

Roll: 2107025

Submitted to

Md Tajmilur Rahman

Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Md. Mubtashim Abrar Nihal

Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

**Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh**

October, 2026

Abstract

This project presents the design and implementation of “Moonlit Forest Campsite,” a real-time procedural 3D scene built entirely using C++ and the OpenGL fixed-function pipeline. The scene portrays a nighttime forest campsite complete with a crackling campfire, procedurally generated trees, an animated owl, exponential-squared fog, drifting smoke particles, rising embers, glowing fireflies, shooting stars, and a dynamic day–night cycle featuring a sunrise transition.

All geometry is generated procedurally at run-time using parametric equations and OpenGL primitives—no external 3D models or image files are loaded. Textures for the ground, tree bark, and tent canvas are synthesized algorithmically using sinusoidal functions combined with uniform random noise. The lighting system employs two OpenGL light sources: a point light representing the campfire with distance-based attenuation, and a directional light for the moon/sun, both of which smoothly transition during the sunrise sequence.

Key rendering techniques demonstrated include multi-layered alpha-blended particle systems (smoke, embers, fireflies), additive blending for fire glow and shooting star streaks, exponential-squared fog (GL_EXP2) for atmospheric depth, procedural terrain with sinusoidal height functions, and real-time camera orbit with mouse and keyboard controls. The project runs at interactive frame rates on standard hardware and is compiled with a single `clang++` command targeting macOS frameworks (OpenGL + GLUT).

To cover the core graphics topics of the course, the scene also implements **hierarchical geometric transformations** (translation, rotation, scaling with the matrix stack), the **Phong reflection model** (ambient, diffuse and specular terms), a run-time switch (key P) between **Gouraud shading** (per-vertex, fixed-function) and **Phong shading** (per-pixel, GLSL), several forms of **motion and animation**, rich keyboard/mouse **interaction**, and an additional **CPU ray tracer** (key G) with shadows, mirror reflection, refraction and fog.

Contents

Abstract	i
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope	1
1.5 Uniqueness of the Project	1
1.6 Applications of the Work	2
1.7 Organisation of the Report	2
1.8 Proposed vs. Implemented Features	2
2 Methodology	4
2.1 Introduction	4
2.2 System Architecture	4
2.3 Camera Model	4
2.4 Geometric Transformations	5
2.5 Viewing and Projection	5
2.6 Terrain Generation	6
2.7 Procedural Texture Generation	6
2.8 Lighting System	7
2.9 Shading Models: Gouraud and Phong	7
2.10 Fog Model	8
2.11 Particle Systems	9
2.12 Fire Rendering	9
2.13 Day–Night Transition	10
2.14 Object Scattering	10
2.15 Ray Tracing	10
3 Implementation	12
3.1 Development Environment	12
3.2 Source Code Structure	12
3.3 Interactive Controls	12
3.4 Scene Screenshots	12
3.5 Shading and Ray-Tracing Results	15
3.6 Code Guide: Functions and Parameters	16
4 Conclusion	18
4.1 Summary	18
4.2 Challenges Faced	18
4.3 Limitations and Future Work	19
References	19

List of Figures

2.1	System Architecture Overview	4
3.1	Scene overview at night	13
3.2	Owl and campfire	14
3.3	Burst embers and tent	14
3.4	Forest, moon and stars	14
3.5	Day–night cycle	15
3.6	Gouraud vs Phong	15
3.7	Ray-traced frames	16

List of Tables

1.1	Proposed features and their final status	2
1.2	Features added beyond the proposal	3
2.1	Gouraud vs. Phong shading in this project	8
3.1	Source File Summary	12
3.2	Keyboard and Mouse Controls	13
3.3	Rasterisation (OpenGL) vs. ray tracing in this project	16
3.4	Important functions	17
3.5	Parameters to change and their effect	17

CHAPTER I

Introduction

1.1 Introduction

Real-time 3D graphics is built from geometric primitives, materials, lighting and camera transformations—concepts that OpenGL exposes directly. **Moonlit Forest Campsite** is a real-time scene of a night-time forest campfire with procedurally generated trees, rocks, a tent and an animated owl, in which the user can orbit the camera, throw logs into the fire and trigger a sunrise.

1.2 Problem Statement

Game engines hide the mathematics of 3D rendering; this project builds a complete scene from scratch using only fixed-function OpenGL, with no external assets.

1.3 Objectives

1. To build a 3D scene from procedural geometry using fixed-function OpenGL.
2. To implement point-light (campfire) and directional (moon/sun) lighting.
3. To demonstrate alpha-blended particles: smoke, embers and fireflies.
4. To implement exponential fog, procedural textures and terrain.
5. To provide interactive controls and a smooth day–night transition.
6. To apply geometric transformations (translate, rotate, scale) hierarchically to build articulated objects such as the owl and trees.
7. To implement and compare Gouraud (per-vertex) and Phong (per-pixel) shading using the ambient–diffuse–specular reflection model.
8. To implement a CPU ray tracer with shadows, reflection and refraction as an extension beyond real-time rasterisation.

1.4 Scope

The scene is implemented in C++17 with OpenGL 2.1 (fixed-function pipeline) and GLUT on macOS.

1.5 Uniqueness of the Project

Unlike single-object tutorials, the project integrates lighting, particles, fog, terrain and animation, all fully procedural, in one interactive scene.

1.6 Applications of the Work

The techniques apply to graphics education, game development, real-time simulation and lightweight embedded graphics.

1.7 Organisation of the Report

Chapter I also compares the proposal with the final result; Chapters II–IV present the methodology (transformations, lighting, Gouraud/Phong shading, animation, ray tracing), the implementation with results and a code guide, and the conclusion.

1.8 Proposed vs. Implemented Features

The project proposal (`proposal.pdf`, “Campfire Forest Night”) promised a Python/Py-OpenGL scene. During development the project was re-implemented in **C++17 with OpenGL 2.1 + GLUT** for speed and so that a GLSL shader and a multi-threaded ray tracer could be added; every proposed feature was kept. Table 1.1 lists the proposed items and their final status.

Table 1.1: Proposed features and their final status

Proposed feature	Status	Where / how
Hierarchical transformations (<code>glPushMatrix/glPopMatrix</code>)	Done	Owl, trees, tripod, tent (<code>draw_owl</code> , <code>draw_tree</code>)
Positional + directional lighting (<code>GL_LIGHT0/GL_LIGHT1</code>), material emission	Done	<code>update_lights</code> , <code>material()</code>
Undulating terrain (44×44 mesh)	Done	<code>terrain_height</code> , <code>draw_ground</code>
6 detailed trees, 15 distant trees, roots, branches, layered foliage with wind sway	Done	<code>draw_tree</code> , <code>draw_distant_forest</code>
Hierarchical owl (body, head, wings, eyes, beak, talons) with animation	Done	<code>draw_owl</code>
Tent with canvas texture, poles, guy ropes	Done	<code>draw_tent</code>
Campfire: 13 stones, 5 logs, 17 ember-bed spheres, 5 additive flame layers	Done	<code>draw_campfire_base</code> , <code>draw_flames</code>
Moon with emissive glow, 90 stars	Done	<code>draw_sky</code>
14 smoke particles, 18 embers, 10 fireflies	Done	<code>draw_smoke</code> , <code>draw_embers</code> , <code>draw_fireflies</code>
52 grass tufts, 19 rocks, 8 soft contact shadows	Done	<code>draw_forest_details</code> , <code>draw_contact_shadows</code>
3 procedural textures (ground, bark, canvas)	Done	<code>TextureBank</code>
Flickering firelight with attenuation	Done	<code>flicker</code> , Eq. 2.13
EXP2 fog with toggle (F)	Done	<code>init_gl</code> , <code>keyboard</code>
Orbit camera: mouse, arrows, zoom, reset	Done	<code>Camera</code> , <code>mouse_motion</code>
Pause/resume (Space), frame-rate independent animation	Done	<code>idle</code> (delta time)
Deterministic seeded randomness	Partial	Scattering is seeded (<code>std::mt19937</code>); shooting stars use <code>rand()</code>
Implementation language Python → C++	Changed	Whole project re-implemented

Beyond the proposal, the following features were added (Table 1.2). The first four were added to satisfy the course requirements for shading and ray tracing.

Table 1.2: Features added beyond the proposal

Extra feature	Details
Gouraud $\leftrightarrow$ Phong shading switch	Key P; GLSL 1.20 per-pixel shader in <code>Shading.cpp</code>
CPU ray tracer	Key G; shadows, reflection, refraction, fog, 2×2 AA, multi-threaded (<code>RayTracer.cpp</code>)
Polished low-poly orb and glass orb	Demonstrate specular highlight and reflection/refraction
On-screen HUD	Shows current shading mode / ray-trace statistics (key H)
Day-night cycle with sun and moon	Key T; all light, fog and sky parameters interpolated
Wind, shooting stars, burst embers	Key W; random streaks; ballistic sparks
Throwable logs, fuel-dependent fire	Key L or mouse click on log pile; fire grows and changes colour
Stumps, hanging pot on tripod, log pile	Additional scene objects (pot sways with wind)
Automatic screenshot mode	<code>./campsite -capture dir</code>

CHAPTER II

Methodology

2.1 Introduction

This chapter describes the mathematical models and algorithmic approaches used to construct each component of the Moonlit Forest Campsite. All formulations are implemented directly in C++ using the OpenGL fixed-function pipeline.

2.2 System Architecture

The application is structured around a single controller class `ForestCampsite` that manages all subsystems:

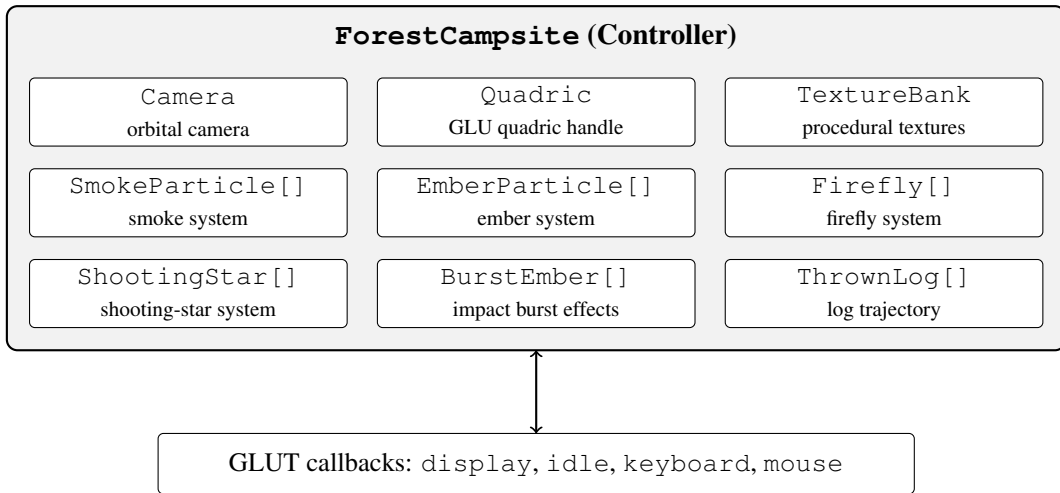


Figure 2.1: System Architecture Overview

2.3 Camera Model

The camera uses a spherical-coordinate orbit model. Given azimuth θ , elevation ϕ , and radius r , the eye position is:

$$\mathbf{e} = \mathbf{t} + r \begin{pmatrix} \cos \phi \sin \theta \\ \sin \phi \\ \cos \phi \cos \theta \end{pmatrix} \quad (2.1)$$

where $\mathbf{t} = (t_x, t_y, t_z)$ is the look-at target. The `gluLookAt` function is then called with $\mathbf{e}$, $\mathbf{t}$, and the up vector $\hat{y} = (0, 1, 0)$.

2.4 Geometric Transformations

All objects are positioned with 4×4 homogeneous matrices applied to column vectors $\mathbf{p} = (x, y, z, 1)^T$:

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.2)$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.3)$$

`glTranslatef`, `glRotatef` and `glScalef` multiply the current matrix on the right, so a call sequence builds $M = TRS$ and a vertex becomes $\mathbf{p}' = M\mathbf{p}$ (the last call acts first on the vertex). `glPushMatrix`/`glPopMatrix` save and restore M , giving *hierarchical* models: a child is drawn relative to its parent,

$$M_{\text{eye}} = M_{\text{tree}} T_{\text{owl}} T_{\text{head}} R_y(\alpha(t)) T_{\text{eye}} \quad (2.4)$$

so that when the head turns by $\alpha(t) = 25^\circ \sin(1.08t)$ the eyes, ears and beak turn with it. Non-uniform scaling is used to create ellipsoids (owl body, rocks, stones) from a unit sphere (`scaled_sphere`). Normals are transformed by the inverse transpose, $\mathbf{n}' = (M^{-1})^T \mathbf{n}$, and re-normalised (`GL_NORMALIZE`). Examples of animated transformations: tree sway $R_z((2.8 + 5w) \sin(1.22t + \phi) + 6w)$, owl wing flap $12^\circ + 14^\circ \sin(4.6t)$, and the thrown log $\mathbf{p}(s) = \mathbf{p}_0(1 - s) + (0, 0.2 + 1.5 \sin(\pi s), 0)s$ with spin $720^\circ s$, $s \in [0, 1]$.

2.5 Viewing and Projection

The model-view matrix is $V \cdot M$ with V from `gluLookAt`. The perspective projection `gluPerspective(51°, a, 0.1, 85)` uses $f = \cot(\text{fov}/2)$:

$$P = \begin{pmatrix} \frac{f}{a} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{z_f + z_n}{z_n - z_f} & \frac{2z_f z_n}{z_n - z_f} \\ 0 & 0 & -1 & 0 \end{pmatrix} \quad (2.5)$$

Visibility is resolved with the depth (Z) buffer (`GL_DEPTH_TEST`); transparent particles disable depth writes (`glDepthMask(GL_FALSE)`). Mouse picking uses `gluUnProject` to cast a ray through the cursor and intersect the ground plane $y = 0$.

2.6 Terrain Generation

The terrain height is a sinusoidal function, blended to zero near the campfire:

$$h(x, z) = [0.16 \sin(0.38x) \cos(0.29z) + 0.07 \sin(0.91x + 0.54z)] \cdot b(r) \quad (2.6)$$

where $r = \sqrt{x^2 + z^2}$ and the clearing blend function is:

$$b(r) = \text{clamp}\left(\frac{r - 3.8}{4.5}, 0, 1\right) \quad (2.7)$$

This keeps the campfire clearing flat while the terrain around it undulates. Surface normals are approximated using finite differences:

$$\hat{n}(x, z) \approx \text{normalise}(h(x - \delta, z) - h(x + \delta, z), 2\delta, h(x, z - \delta) - h(x, z + \delta)) \quad (2.8)$$

with step size $\delta = 0.08$. The ground is tessellated into a 44×44 grid of triangle strips covering a 32×32 unit area.

2.7 Procedural Texture Generation

Three textures are generated algorithmically without loading any image files:

2.7.1 Ground Texture

A 96×96 pixel texture combining multiple sine waves with random noise to simulate a dirt/grass surface. Colour values are modulated by:

$$c(u, v) = c_{\text{base}} + A \sin(f_1 u) \cos(f_2 v) + \eta \quad (2.9)$$

where $\eta \sim \mathcal{U}(-\epsilon, \epsilon)$ is uniform random noise.

2.7.2 Bark Texture

A 64×96 pixel texture using vertical sinusoidal streaks with horizontal perturbation:

$$c(u, v) = c_{\text{bark}} + A_1 \sin(f_v \cdot v + A_2 \sin(f_u \cdot u)) \quad (2.10)$$

2.7.3 Canvas Texture

A 64×64 pixel texture with a woven cross-hatch pattern created by modulating horizontal and vertical sine functions.

2.8 Lighting System

The lighting follows the **Phong reflection model**: the colour at a surface point is the sum of an emissive term, a global ambient term, and for every light an ambient, a diffuse and a specular component, attenuated with distance:

$$I = I_e + I_a k_a + \sum_i \frac{1}{k_c + k_l d_i + k_q d_i^2} \left[I_{a,i} k_a + I_{d,i} k_d \max(\hat{N} \cdot \hat{L}_i, 0) + I_{s,i} k_s \max(\hat{R}_i \cdot \hat{V}, 0)^{n_s} \right] \quad (2.11)$$

where $\hat{N}$ is the surface normal, $\hat{L}_i$ the unit vector to the light, $\hat{V}$ the unit vector to the viewer, and the reflection vector is

$$\hat{R} = 2(\hat{N} \cdot \hat{L})\hat{N} - \hat{L} \quad (2.12)$$

I_a, I_d, I_s are the ambient, diffuse and specular light intensities, k_a, k_d, k_s the material coefficients, k_c, k_l, k_q the attenuation constants, n_s the shininess exponent and I_e the material emission. OpenGL's fixed-function pipeline replaces $\hat{R} \cdot \hat{V}$ by the Blinn half-vector form $(\hat{N} \cdot \hat{H})^{n_s}$ with $\hat{H} = (\hat{L} + \hat{V}) / \|\hat{L} + \hat{V}\|$; our Phong GLSL shader uses the original $\hat{R} \cdot \hat{V}$ of Eq. 2.12. `GL_COLOR_MATERIAL` makes $k_a = k_d =$ the current `glColor`; the specular coefficient and shininess are set by `material()` (e.g. the polished orb uses $k_s = 0.95, n_s = 70$). Two light sources illuminate the scene:

1. **Light 0 (Campfire – Point Light)**: Positioned at $(0, 1.08, 0)$ with attenuation parameters $k_c = 1.0, k_l = 0.075, k_q = 0.026$. Its intensity is modulated by a flicker function:

$$F(t, \varphi) = 0.54 \sin(12.7t + \varphi) + 0.29 \sin(21.3t + 2.1\varphi) + 0.17 \sin(37.9t + 0.7\varphi) \quad (2.13)$$

2. **Light 1 (Moon/Sun – Directional Light)**: Direction vector interpolated between moonlight $(0.24, 0.82, 0.18)$ and sunlight $(0, 1, 1)$ based on `time_of_day` $\tau \in [0, 1]$:

$$\mathbf{L}_1 = (1 - \tau)\mathbf{L}_{\text{moon}} + \tau\mathbf{L}_{\text{sun}} \quad (2.14)$$

2.9 Shading Models: Gouraud and Phong

The reflection model of Eq. 2.11 gives a colour at *one point*; a *shading model* decides where it is evaluated. For a triangle with vertices $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ and a pixel with barycentric coordinates (α, β, γ) , $\alpha + \beta + \gamma = 1$:

- **Flat**: one colour per polygon from the face normal (not used).

- **Gouraud (per-vertex):** the reflection model is evaluated at the three vertices using vertex normals, and the *colours* are interpolated:

$$I(\mathbf{p}) = \alpha I(\mathbf{v}_1) + \beta I(\mathbf{v}_2) + \gamma I(\mathbf{v}_3) \quad (2.15)$$

- **Phong (per-pixel):** the *normals* are interpolated and the reflection model is evaluated for every pixel:

$$\hat{N}(\mathbf{p}) = \frac{\alpha \hat{N}_1 + \beta \hat{N}_2 + \gamma \hat{N}_3}{\|\alpha \hat{N}_1 + \beta \hat{N}_2 + \gamma \hat{N}_3\|}, \quad I(\mathbf{p}) = I(\hat{N}(\mathbf{p})) \text{ from Eq. 2.11} \quad (2.16)$$

Because Gouraud interpolates intensities, a specular highlight that falls *inside* a triangle is lost or smeared, and a point light close to a large polygon (the campfire light on the 0.73-unit ground cells or on the tent canvas) produces a faceted hot-spot. Phong shading reproduces a smooth highlight at higher cost (per-pixel evaluation).

Table 2.1: Gouraud vs. Phong shading in this project

	Gouraud	Phong
Lighting evaluated	Per vertex	Per pixel (fragment)
Interpolated quantity	Colour	Normal (and eye position)
Implementation	Fixed function, <code>glShadeModel (GL_SMOOTH)</code>	GLSL 1.20 program in <code>Shading.cpp</code>
Specular term	Half-vector $(N \cdot H)^n$	Reflection vector $(R \cdot V)^n$
Key	P (off)	P (on)

The Phong program re-uses the same OpenGL state (`gl_LightSource`, `gl_FrontMaterial`, `gl_Fog`, `glColor`, current texture), so all existing draw calls work unchanged in both modes. Because GLSL cannot see whether lighting/texturing/fog is enabled, `glEnable`/`glDisable` are wrapped by two small hook functions that upload three boolean uniforms (`uLighting`, `uTexture`, `uFog`).

2.10 Fog Model

The scene uses GL_EXP2 (exponential-squared) fog:

$$f = e^{-(d \cdot \rho)^2} \quad (2.17)$$

where d is the distance from the eye to the fragment and ρ is the fog density. The final fragment colour is blended as:

$$C_{\text{final}} = f \cdot C_{\text{fragment}} + (1 - f) \cdot C_{\text{fog}} \quad (2.18)$$

2.11 Particle Systems

2.11.1 Smoke Particles

Each of the 14 smoke particles follows a cyclic life with phase offset:

$$\ell_i = (p_i + t \cdot s_i) \bmod 1 \quad (2.19)$$

Horizontal drift uses curl-like motion:

$$x_i(\ell) = x_{\text{off}} + \sin(0.72t + \psi_i + 3\ell) \cdot (0.10 + 0.44\ell) + w \cdot \ell \cdot 2 \quad (2.20)$$

Opacity decays as $\alpha = 0.22 \cdot (1 - \ell)^{1.4}$.

2.11.2 Ember Particles

18 embers spiral upward with increasing radius:

$$\begin{cases} \theta_i(\ell) &= \theta_0 + t \cdot d_i + 2.4\ell \\ r_i(\ell) &= r_0(0.4 + \ell) \\ y_i(\ell) &= 0.75 + 2.6\ell \end{cases} \quad (2.21)$$

2.11.3 Firefly Motion

Each of the 10 fireflies orbits an elliptical path:

$$\mathbf{p}_i(t) = \mathbf{c}_i + \begin{pmatrix} r_{x,i} \cos(\omega_i t + \phi_i) \\ 0.22 \sin(1.7(\omega_i t + \phi_i)) \\ r_{z,i} \sin(\omega_i t + \phi_i) \end{pmatrix} \quad (2.22)$$

with a pulsating glow: $\alpha = 0.72 + 0.28 \sin(4.2t + \phi_i)$.

2.12 Fire Rendering

The campfire is rendered using five nested cones with additive blending:

$$C_{\text{flame}} = C_{\text{src}} \cdot \alpha_{\text{src}} + C_{\text{dst}} \cdot 1 \quad (2.23)$$

Each cone “leans” dynamically:

$$\theta_{\text{lean}} = 4 \sin(7t + \phi) + 20w \quad (2.24)$$

where w is the wind strength $\in [0, 1]$.

2.13 Day–Night Transition

All environmental parameters are linearly interpolated using `time_of_day` τ :

$$P(\tau) = (1 - \tau) \cdot P_{\text{night}} + \tau \cdot P_{\text{day}} \quad (2.25)$$

Applied to: background colour, ambient light, fog colour, fog density, moonlight/sunlight direction, and star opacity.

2.14 Object Scattering

Rocks and grass tufts are scattered using a rejection-sampling algorithm with a Mersenne Twister PRNG:

$$(x_i, z_i) = (r_i \cos \alpha_i, r_i \sin \alpha_i), \quad r_i \sim \mathcal{U}(r_{\text{in}}, r_{\text{out}}), \quad \alpha_i \sim \mathcal{U}(0, 2\pi) \quad (2.26)$$

Subject to: $\sqrt{(x_i - 3.2)^2 + (z_i - 2.5)^2} > 2.0$ (clearing exclusion zone).

2.15 Ray Tracing

In addition to real-time rasterisation, a CPU ray tracer (Whitted style) renders the same scene when the user presses G. A ray is $\mathbf{r}(t) = \mathbf{o} + t\hat{\mathbf{d}}$, $t > 0$. For pixel (x, y) of a $W \times H$ image with camera basis $(\hat{\mathbf{f}}, \hat{\mathbf{r}}, \hat{\mathbf{u}})$ the primary ray direction is

$$\hat{\mathbf{d}} = \text{normalise}(\hat{\mathbf{f}} + u\hat{\mathbf{r}} + v\hat{\mathbf{u}}), \quad u = \left(\frac{2(x+0.5)}{W} - 1 \right) a \tan \frac{\theta}{2}, \quad v = \left(1 - \frac{2(y+0.5)}{H} \right) \tan \frac{\theta}{2} \quad (2.27)$$

with $a = W/H$ and $\theta = 51^\circ$ (the same as the OpenGL camera).

2.15.1 Ray–Object Intersection

Plane $y = y_0$: $t = (y_0 - o_y)/d_y$.

Sphere / ellipsoid (centre $\mathbf{c}$, radii $\mathbf{r}$): the ray is divided component-wise by $\mathbf{r}$ so the surface becomes the unit sphere, $\|\mathbf{o}' + t\mathbf{d}'\|^2 = 1$:

$$A = \mathbf{d}' \cdot \mathbf{d}', \quad B = \mathbf{o}' \cdot \mathbf{d}', \quad C = \mathbf{o}' \cdot \mathbf{o}' - 1, \quad t = \frac{-B \mp \sqrt{B^2 - AC}}{A} \quad (2.28)$$

and the normal is $\hat{N} \propto (\mathbf{p} - \mathbf{c})/\mathbf{r}^2$ (component-wise).

Cylinder / cone (frustum) along the local y -axis, radius $r(y) = r_0 + ky$, $k = (r_1 - r_0)/h$: $x^2 + z^2 = r(y)^2$ gives

$$A = d_x^2 + d_z^2 - (kd_y)^2, \quad B = o_x d_x + o_z d_z - r_o k d_y, \quad C = o_x^2 + o_z^2 - r_o^2, \\ r_o = r_0 + k o_y, \quad t = \frac{-B \pm \sqrt{B^2 - AC}}{A} \quad (2.29)$$

accepted when $0 \leq y \leq h$; the two end caps are discs ($y = 0, y = h$). The normal is $(p_x, -kr(y), p_z)$ normalised. Trees, tripod legs, logs and branches are all frusta placed with an arbitrary axis.

Triangle (tent, owl beak) by Möller–Trumbore with edges $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$, $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$, $\mathbf{p} = \hat{\mathbf{d}} \times \mathbf{e}_2$, $\mathbf{s} = \mathbf{o} - \mathbf{v}_0$:

$$\det = \mathbf{e}_1 \cdot \mathbf{p}, \quad u = \frac{\mathbf{s} \cdot \mathbf{p}}{\det}, \quad v = \frac{\hat{\mathbf{d}} \cdot (\mathbf{s} \times \mathbf{e}_1)}{\det}, \quad t = \frac{\mathbf{e}_2 \cdot (\mathbf{s} \times \mathbf{e}_1)}{\det} \quad (2.30)$$

with a hit when $u \geq 0, v \geq 0, u + v \leq 1$.

2.15.2 Shading, Shadows, Reflection and Refraction

At the nearest hit $\mathbf{p}$ the colour is the Phong model of Eq. 2.11, but each light contributes only if a **shadow ray** from $\mathbf{p} + \epsilon \hat{N}$ towards the light is not blocked (hard shadows). For mirror surfaces the colour is mixed with the traced reflected ray, and for glass with the reflected and refracted rays:

$$\hat{\mathbf{r}} = \hat{\mathbf{d}} - 2(\hat{\mathbf{d}} \cdot \hat{N})\hat{N}, \quad C = (1 - k_r)C_{\text{local}} + k_r C_{\text{reflect}} \quad (2.31)$$

$$\hat{\mathbf{t}} = \eta \hat{\mathbf{d}} + \left(\eta \cos \theta_i - \sqrt{1 - \eta^2(1 - \cos^2 \theta_i)} \right) \hat{N}, \quad \eta = \frac{n_1}{n_2}, \quad \cos \theta_i = -\hat{\mathbf{d}} \cdot \hat{N} \quad (2.32)$$

If the radicand is negative the ray is totally internally reflected. The share of reflected light is the Schlick approximation of the Fresnel equations,

$$F = F_0 + (1 - F_0)(1 - \cos \theta_i)^5, \quad F_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2, \\ C_{\text{glass}} = (1 - T)C_{\text{local}} + T(F C_{\text{reflect}} + (1 - F) C_{\text{refract}}) \quad (2.33)$$

with glass index $n = 1.5$. Recursion stops at depth 4. Fog uses the same law as OpenGL, $f = e^{-(\rho t)^2}$, with t the distance along the ray. Anti-aliasing uses 2×2 super-sampling, bounding spheres reject most objects quickly, and image rows are distributed over all CPU cores (`std:::thread`). The cost is $O(\text{pixels} \times \text{samples} \times \text{rays per sample} \times \text{objects})$, which is why the ray tracer renders a still frame rather than running in real time.

CHAPTER III

Implementation

3.1 Development Environment

The project was developed on macOS using `clang++` with C++17. The build command is:

```
1 clang++ -std=c++17 -O2 -framework OpenGL -framework GLUT \  
2   main.cpp Scene.cpp Primitives.cpp Particles.cpp Shading.cpp  
   RayTracer.cpp -o campsite
```

Listing 3.1: Build Command

No third-party libraries are required. The total source code is 3,091 lines across thirteen files (Table 3.1), of which six `.cpp` files are compiled by the build command.

3.2 Source Code Structure

Table 3.1 summarises the source files of the project.

Table 3.1: Source File Summary

File	Lines	Purpose
<code>main.cpp</code>	8	Entry point; instantiates <code>ForestCampsite</code>
<code>Scene.h</code>	137	Class declarations: <code>Camera</code> , <code>ForestCampsite</code> , <code>structs</code>
<code>Scene.cpp</code>	1,917	All rendering logic, input handling, animation, HUD, ray-scene builder
<code>Primitives.h/cpp</code>	254	<code>Quadric</code> , <code>TextureBank</code> , geometric helpers
<code>Particles.h/cpp</code>	95	Particle data generation (smoke, embers, fireflies)
<code>Shading.h/cpp</code>	206	GLSL Phong shader, Gouraud/Phong switch, <code>glEnable</code> hooks
<code>RayTracer.h/cpp</code>	475	Ray tracer: intersections, Phong, shadows, reflection, refraction
Total	3,091	13 files

3.3 Interactive Controls

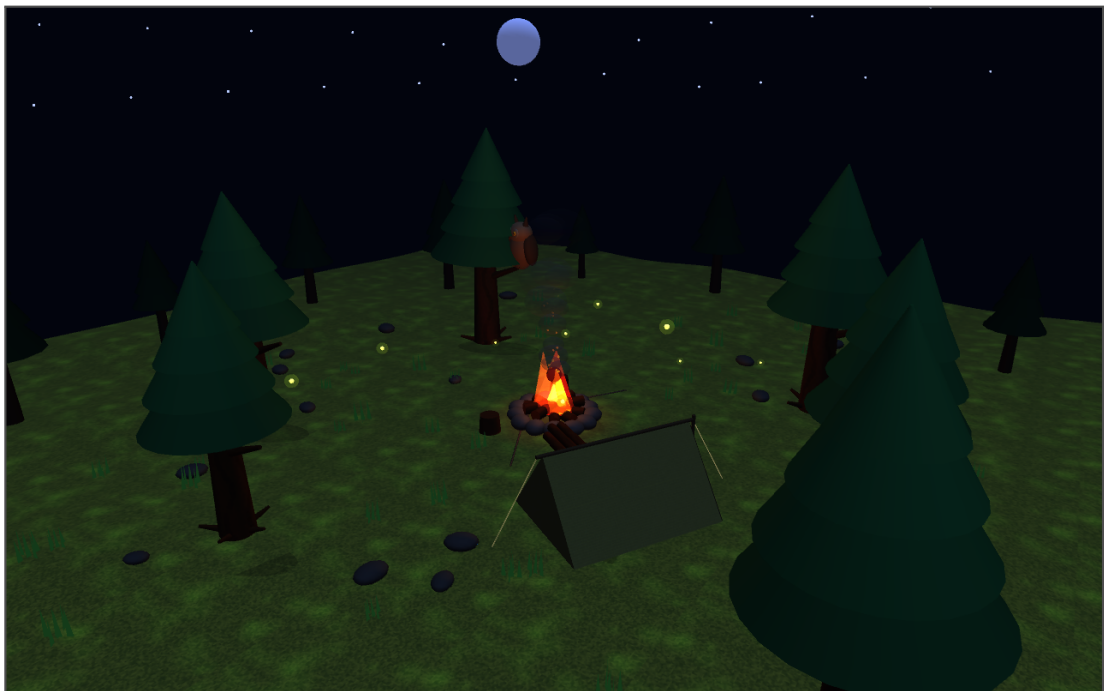
Table 3.2 lists the keyboard and mouse controls supported by the application.

3.4 Scene Screenshots

Figure 3.1 shows the complete night scene, Figures 3.2–3.4 show individual scene elements, and Figures 3.5–3.5 show the day–night transition.

Table 3.2: Keyboard and Mouse Controls

Input	Action
Mouse Drag	Orbit camera (azimuth + elevation)
Arrow Keys	Orbit camera (keyboard alternative)
+ / -	Zoom in / out
L	Throw a log into the campfire
W	Toggle wind effect
T	Toggle sunrise/sunset transition
Space	Pause / resume animation
F	Toggle fog on/off
R	Reset camera to default position
P	Toggle Gouraud (per-vertex) ↔ Phong (per-pixel) shading
G	Render a ray-traced frame of the current view / return to real-time view
H	Show / hide the HUD panel
Mouse click on log pile	Throw a log (picking with <code>gluUnProject</code>)
Q / Esc	Quit application

**Figure 3.1:** Complete scene overview at night – campfire, tent, trees, owl, moon and stars are visible.

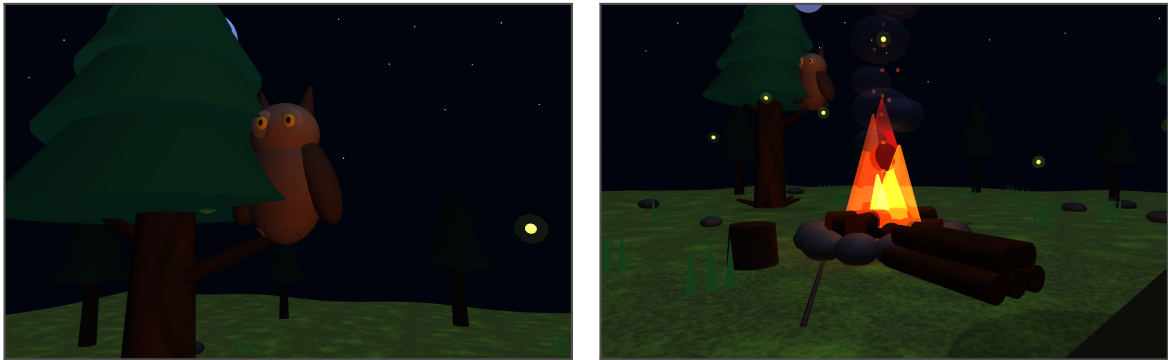


Figure 3.2: Left: animated owl on a tree branch (bobbing, head turn, wing flap). Right: multi-layered campfire with additive flame cones, embers and stone ring.

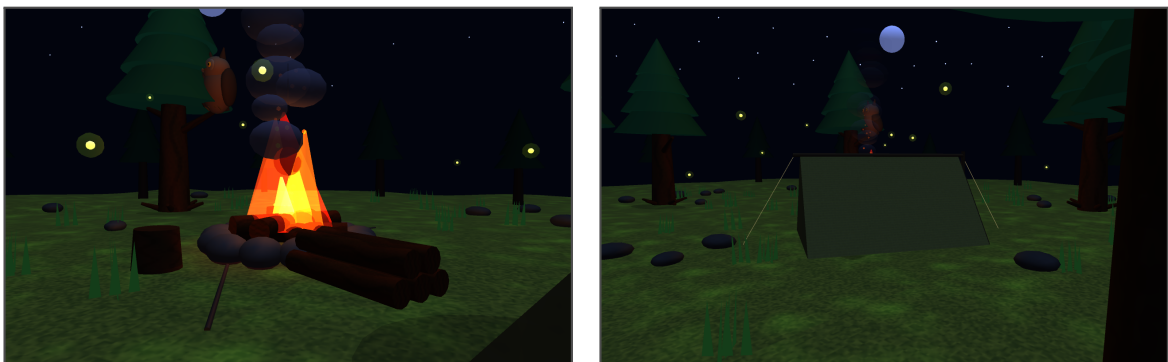


Figure 3.3: Left: burst embers with ballistic trajectories after a log is thrown into the fire. Right: A-frame tent with textured canvas, built from quads, triangles and `GL_LINES` guy-ropes.

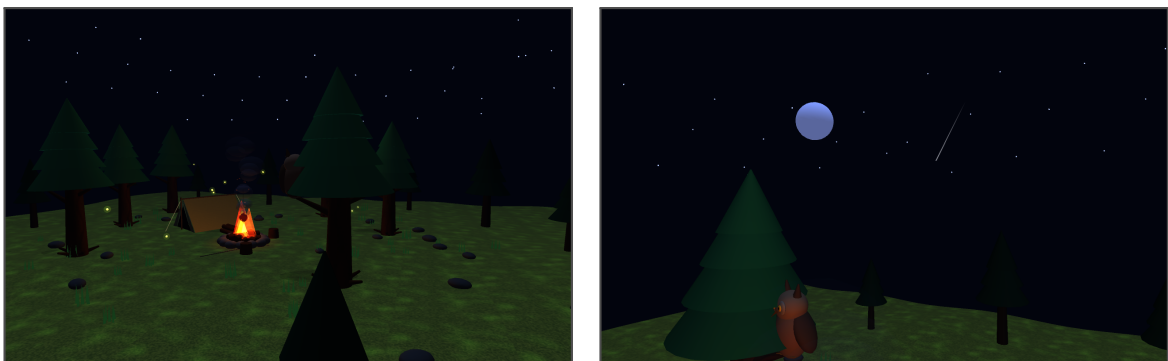


Figure 3.4: Left: wide-angle forest view with four-layer conical foliage and the distant forest ring. Right: moon as an emissive solid sphere with stars rendered as `GL_POINTS`.

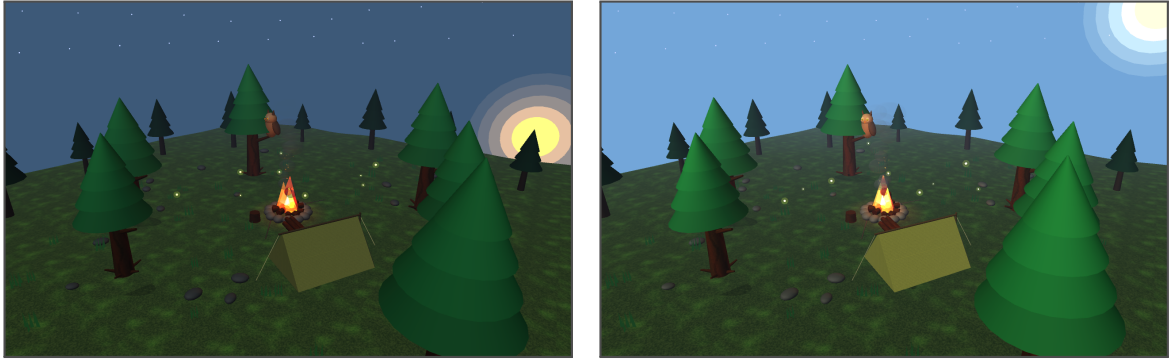


Figure 3.5: Day–night cycle: mid-sunrise with additive glow halos (left) and full daylight after all lighting, fog and sky parameters have completed the transition (right).

3.5 Shading and Ray-Tracing Results

Figure 3.6 compares Gouraud and Phong shading on the same frame (switch with P). The low-poly polished orb (12×8 faces, $k_s = 0.95$, $n_s = 70$) shows the clearest difference: with Gouraud the highlight is lost between vertices, with Phong a sharp, smooth highlight appears; on the ground the firelight hot-spot is also smoother.

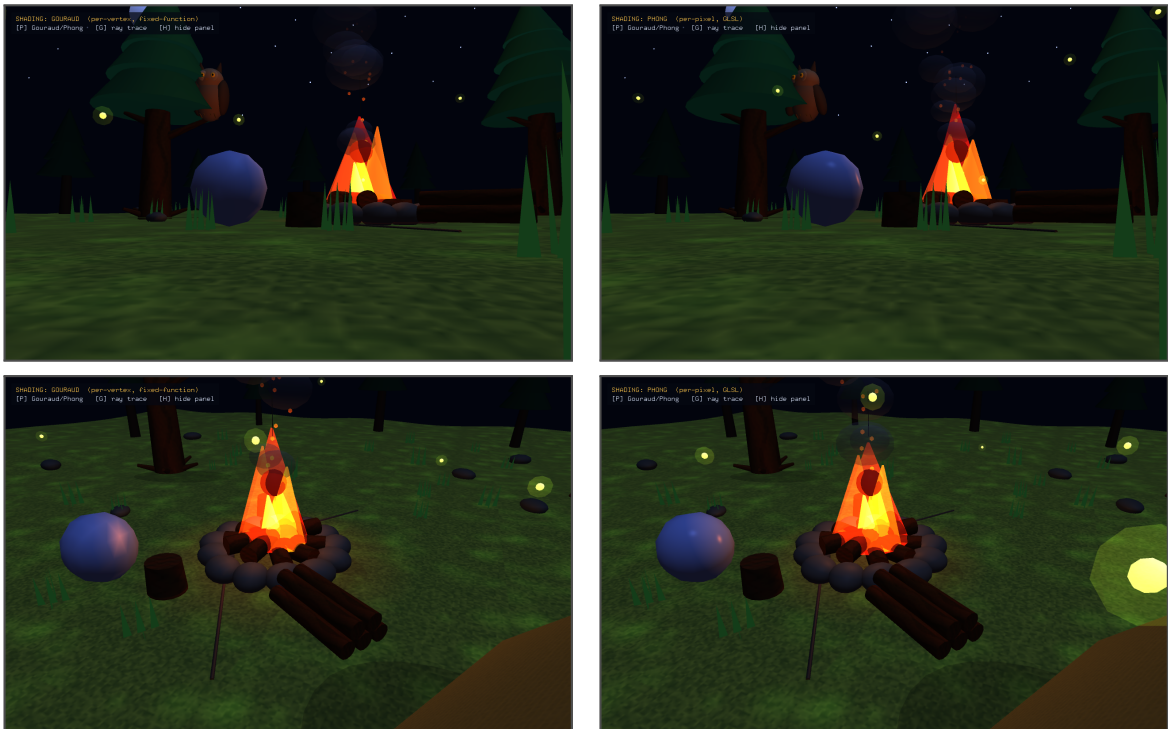


Figure 3.6: Gouraud (left) vs. Phong (right) shading: polished orb (top) and campfire light on the ground (bottom).

Figure 3.7 shows the ray-traced frames: shadows of the trees on the ground, the mirror orb reflecting the scene, the glass orb refracting the flames (inverted image), and the owl, tent and campfire built from spheres, cones, cylinders and triangles. The scene of 195 primitives renders at 960×600 with 2×2 anti-aliasing in about 0.6 s on the development machine.

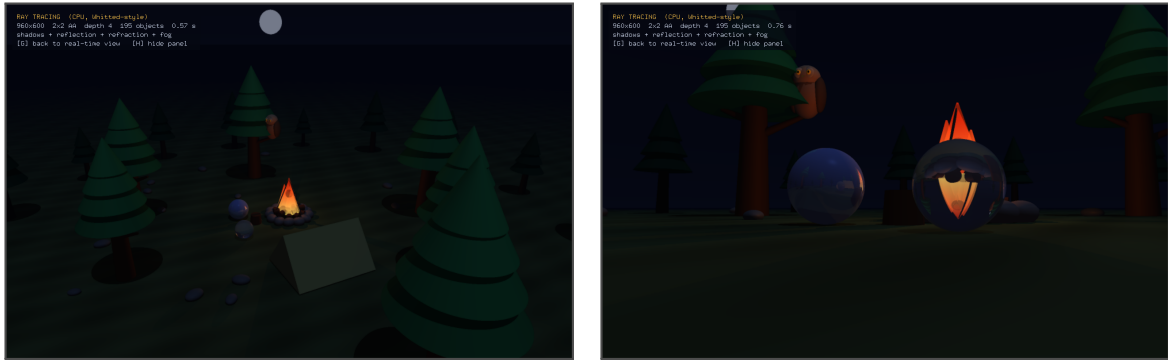


Figure 3.7: Ray tracing (key G): overview with hard shadows (left); mirror and glass orbs with reflection and refraction (right).

Table 3.3: Rasterisation (OpenGL) vs. ray tracing in this project

	Real-time OpenGL	Ray tracer (G)
Shadows	Soft blob contact shadows (fake)	True hard shadows via shadow rays
Reflection / refraction	None	Mirror orb, glass orb
Particles, textures, animation	Yes	No (still frame, flat ground, flat colours)
Speed	60 frames/s	≈ 0.6 s per frame

3.6 Code Guide: Functions and Parameters

Table 3.4 summarises what each important function does, and Table 3.5 lists parameters that can be changed to see an immediate effect.

Table 3.4: Important functions

Function (file)	Purpose
init_gl (Scene.cpp)	Enables depth test, lighting, GL_SMOOTH, fog; creates textures and the Phong shader
update_lights	Sets position/colour of fire light and moon/sun light, ambient and fog every frame
display	Per frame: camera, lights, selects shading mode, draws all objects, HUD; starts ray trace
idle	Delta-time animation update (time, fuel, wind, day-night, stars, logs, embers)
keyboard, special_key, mouse, mouse_motion	User interaction (Table 3.2)
draw_tree, draw_owl	Hierarchical models using glPushMatrix/glRotatef
draw_flames, draw_smoke	Additive / alpha blended animated particles and cones
material (Primitives.cpp)	Sets k_a, k_d, k_s, n_s , emission and colour
shading_use, fragment shader (Shading.cpp)	Switch Gouraud/Phong; per-pixel ambient + diffuse + specular + fog
build_ray_scene (Scene.cpp)	Converts the current scene state, lights and fog to ray-tracer objects
RayTracer::trace, intersect, render	Recursive colour, ray-object tests, multi-threaded image loop

Table 3.5: Parameters to change and their effect

Parameter (location)	Effect when changed
GL_FOG_DENSITY (init_gl, update_lights)	Larger value: denser fog, shorter visibility
Light 0 attenuation k_c, k_l, k_q (init_gl)	Larger k_l, k_q : fire light reaches a shorter distance
l0_pos (update_lights)	Moves the fire light; changes highlights and shading on all objects
shininess / specular in material() calls	Higher shininess: smaller, sharper highlight (try the orb in draw_demo_orb)
glutSolidSphere(0.45, 12, 8) (orb)	More slices/stacks: Gouraud approaches Phong quality
delta * 0.05f (idle)	Speed of the sunrise (0.05 = 20 s)
wind_strength factors (idle)	Wind ramp speed and tree/flame lean
Camera() constructor	Initial azimuth, elevation, distance
rt.render(cam, env, rt_w, rt_h, 2, 4)	Anti-aliasing (2), recursion depth (4), resolution (960 px)
reflectivity, transparency, ior (build_ray_scene)	Mirror strength, glass opacity and refraction index

CHAPTER IV

Conclusion

4.1 Summary

This project demonstrates that a visually rich, interactive 3D scene can be built entirely from procedural primitives using OpenGL, and that the same scene can be rendered by an independent ray tracer. The Moonlit Forest Campsite incorporates:

- Over 20 distinct object types (trees, tent, owl, campfire, rocks, grass, stumps, tripod, logs)
- Hierarchical transformations (translate, rotate, scale with the matrix stack)
- Phong reflection model with a point and a directional light, and **Gouraud and Phong shading** switchable at run time
- Three procedurally generated textures (ground, bark, canvas)
- Three continuous particle systems (smoke, embers, fireflies) and event-driven effects (burst embers, shooting stars)
- A complete day–night cycle with smooth parameter interpolation
- Interaction: camera orbit/zoom, log throwing (key and mouse picking), wind, fog, sunrise, pause, shading and ray-trace switches
- A CPU **ray tracer** with shadows, mirror reflection, refraction and fog

All features promised in the proposal were delivered (one, deterministic randomness, only partially), and four features were added on top.

4.2 Challenges Faced

1. **Blending Order:** Correct transparency needed careful control of depth writes (`glDepthMask`) and blending mode (additive vs. alpha).
2. **Lighting Transitions:** Avoiding visual “popping” required interpolating attenuation, ambient, diffuse, specular and fog together.
3. **Per-pixel shading on a fixed-function code base:** The GLSL shader had to reproduce the fixed-function state (lights, material, texture, fog); this was solved by reading the built-in uniforms and hooking `glEnable`/`glDisable`.
4. **Ray-tracer scene:** The fire light lies inside the cooking pot and flames, so these objects do not cast shadows; ray-traced flames use transparency to show the inner layers.

4.3 Limitations and Future Work

- The ray tracer shows a still frame with a flat ground and without particles, textures or soft shadows; a BVH, soft shadows and textures could be added.
- **Shadow Mapping:** Real-time shadow casting from the campfire for the OpenGL view.
- **Water Body:** A reflective pond near the campsite using environment mapping.
- **Sound Integration:** Ambient nature sounds and crackling fire audio using OpenAL.
- **Weather Effects:** Rain, snow, and cloud cover with corresponding lighting changes.

References

1. The report structure and formatting are based on the provided template `Final_CSE-3200_report_template.pdf`.